

Recursive Log Diagnosis with ProcessIR:

A DLR-Lambda Experiment on Loghub

Nikit Phadke

June 2026

Abstract

Operational logs often contain a failure’s causal evidence, but the evidence is rarely adjacent, self-contained, or expressed as an explicit root cause. A local subsystem may report success while the parent process has not reached a valid terminal state. This paper reports a standalone experiment combining a lambda-style recursive execution plan, inspired by lambda-RLM [1], with Disentangled Language Representation (DLR) ProcessIR diagnosis. The experiment uses the public Loghub Linux log sample as a real noisy substrate and injects five distant synthetic evidence events that encode a stale-policy failure. A direct first-window parser fails to recover the incident. A naive chunk event merge recovers the observed events but misses the latent stale-representation root cause. The DLR-lambda pipeline, implemented as deterministic split, map, filter, and reduce stages, recovers all event nodes, the latent condition, all causal edges, the root cause, the false terminal state, and a corrective prescription. The result is not presented as a general benchmark score. It is a falsification-style demonstration that typed recursive control and typed causal intermediate representations solve different parts of the log diagnosis problem: recursion makes distributed evidence tractable, while ProcessIR makes the reduced result semantically inspectable.

1 Introduction

Many incident traces are not failures of a single action. They are failures of composition. One component reports a local terminal state, while another component later reveals that the parent workflow was never globally complete. This pattern is common in release pipelines, account workflows, payments, compliance processes, and agentic tool-use systems. A logging system may therefore contain all required evidence while still making the incident hard to diagnose: the relevant lines are separated by noise, time, service boundaries, and representation boundaries.

The experiment in this paper asks a narrow question:

Can a recursive, lambda-style execution plan recover a typed DLR ProcessIR diagnosis from distributed evidence embedded in realistic logs?

The implemented answer is yes for the constructed probe. The result is valuable because it separates two problems that are often collapsed. Lambda-RLM argues that typed symbolic control can make long-context recursive reasoning more verifiable and efficient than open-ended recursive code generation [1]. This experiment asks what typed recursive control should compose *into* for incident diagnosis. First, there is a *control problem*: how to scan and compose evidence across a long sequence without assuming it is adjacent. Second, there is a *semantic target problem*: what structure should the diagnosis produce once the evidence is found? The proposed pipeline addresses the first problem with typed recursive operators and the second with ProcessIR.

The contribution is an executable micro-benchmark over Loghub Linux logs [2, 3, 4] that combines lambda-RLM-style recursive control with a DLR ProcessIR target. The benchmark includes three systems: a direct first-window parser, a naive chunk event merge, and the DLR-lambda ProcessIR reducer. Only the third system recovers the latent root cause and the full causal graph.

2 Background

2.1 Lambda-RLM

Lambda-RLM replaces free-form recursive control code with a typed functional runtime grounded in lambda calculus [1]. In that setting, recursive reasoning is expressed through a compact library of verified combinators, and neural inference is applied to bounded leaf subproblems. This paper does not reimplement the full lambda-RLM system. It borrows the core control idea: make long-context reasoning a typed recursive program rather than an unstructured prompt over an entire context.

The difference is the output target. Lambda-RLM primarily addresses reliable long-context control. DLR ProcessIR addresses typed semantic diagnosis. The experiment tests their combination: recursive control over log chunks, followed by reduction into a causal process graph.

2.2 Loghub as Noisy Substrate

Loghub is a public collection of system log datasets for AI-driven log analytics research [2, 3]. Its repository describes datasets from distributed systems, operating systems, mobile systems, server applications, and standalone software. The experiment uses the Linux log sample, which is small enough to download and run locally while still providing a real background of unstructured operating-system log lines.

The benchmark does not claim that the original Loghub Linux sample contains the target incident. Instead, it uses Loghub as the noisy carrier and injects a controlled synthetic incident. This lets the experiment keep a known gold graph while testing whether the parser can recover evidence distributed across a real log stream.

2.3 ProcessIR

ProcessIR is the process-level intermediate representation used by the DLR experiments. The relevant fields for this paper are:

- **event frames**: observed events with roles such as false-terminal state, invariant violation, and terminal outcome;
- **latent conditions**: inferred structural causes that are not themselves single log events;
- **causal graph**: typed edges between event frames and latent conditions;
- **diagnosis**: motif-level explanation of the failure;
- **prescription**: corrective action at the motif level;
- **confidence**: calibrated confidence for the diagnosis.

The key semantic distinction is local versus global completion. A local transition can be successful and still be globally invalid if it violates a parent invariant or is based on a stale representation.

3 Experimental Design

3.1 Incident Construction

The experiment injects five evidence lines into `Linux_2k.log`. The final trace has 2005 lines. Evidence is intentionally separated across chunks so that a single-window parser cannot see the incident and a shallow event merge lacks the pressure to infer the hidden precondition.

Table 1: Injected distributed evidence.

Line	Evidence summary
121	Policy cache has a long TTL and is treated as the local policy source.
482	Authoritative policy changes to version v18 and now requires manager review.
953	Worker reports local success and approves refund R-913 using cached policy v17.
1304	Audit detects invariant violation: expected v18, observed v17, manager review missing.
1705	Finance blocks settlement because the approval was not a valid terminal state.

The gold root cause is not any single event. It is the latent condition: *the worker approved a refund using stale cached policy v17 after the authoritative policy changed to v18*. This is a representation and decay failure that produces a false terminal state.

3.2 Recursive Execution Plan

The DLR-lambda execution plan is deliberately simple:

```

SPLIT(log_lines, chunk_size=350)
MAP(parse_chunk_to_event_frames)
FILTER(relevant_incident_frames)
REDUCE(infer_latent_conditions + merge_causal_graph)
OUTPUT(ProcessIR + prescription)

```

The important operation is the reduce step. The reduce stage composes local evidence into a latent condition and typed causal graph. It is therefore not merely concatenating parsed events.

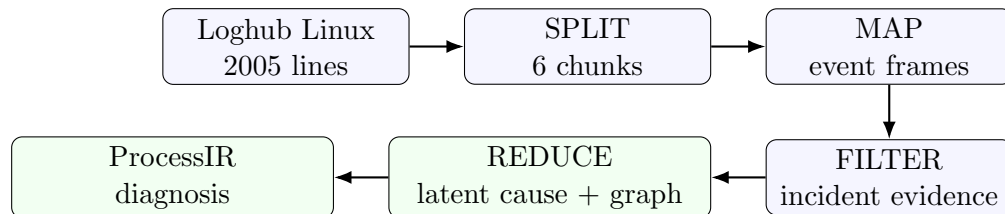


Figure 1: DLR-lambda execution plan. Recursive control finds distributed evidence; ProcessIR gives the reduction a typed semantic target.

3.3 Systems Compared

The benchmark compares three systems:

1. **Direct first-window parser:** runs a DLR-style parser on the first chunk only. This tests what happens when the diagnosis has access to local context but not the distributed trace.

2. **Naive chunk event merge**: parses all chunks and merges observed event frames, but does not infer latent conditions during reduction.
3. **DLR-lambda ProcessIR**: parses chunks, filters incident evidence, and reduces evidence into latent conditions, causal edges, root cause, and prescription.

4 Gold ProcessIR

The gold graph contains three observed event nodes, one latent condition, and three typed causal edges.

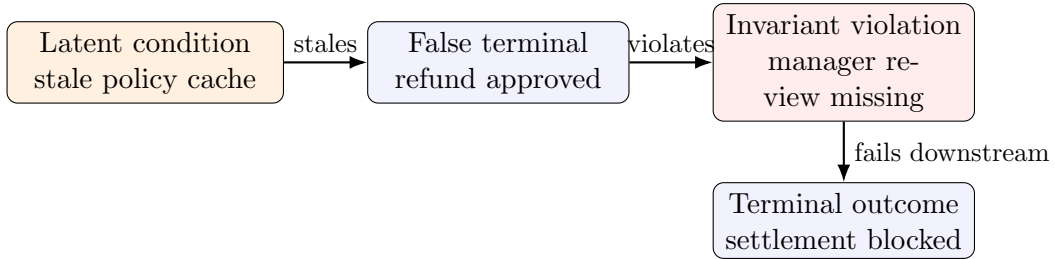


Figure 2: Gold causal graph for the injected incident.

5 Results

Table 2: System-level scores.

System	Conf.	Event recall	Latent recall	Edge recall	Root cause	False terminal	L/G sep.	Result
Direct first-window parser	0.86	0.00	0.00	0.00	0.00	no	no	FAIL
Naive chunk event merge	0.78	1.00	0.00	0.67	0.00	yes	yes	FAIL
DLR-lambda ProcessIR	0.86	1.00	1.00	1.00	1.00	yes	yes	PASS

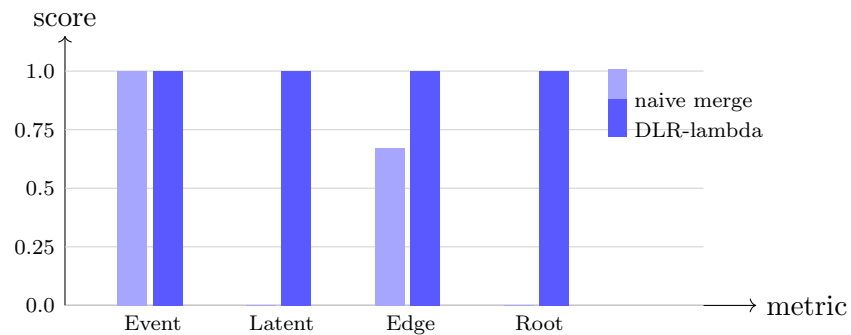


Figure 3: The naive merge sees observed events but misses the latent root cause. The DLR-lambda reducer recovers both observed and latent structure.

5.1 Interpretation

The direct first-window parser fails because it observes only the early configuration evidence. This is expected. The incident is not locally present in one chunk.

The naive chunk event merge is more interesting. It sees all observed incident events, detects the false terminal state, and separates local from global status. However, it does not infer the latent stale-policy condition. It also misses the causal edge from stale representation to false terminal approval. This is the critical failure mode: without a typed reduction step, event recall can look perfect while root-cause accuracy remains zero.

The DLR-lambda ProcessIR system recovers the latent condition and all causal edges:

Table 3: Recovered ProcessIR elements.

Element	Type	Meaning
<code>refund_approved_false_terminal</code>	Event frame	A worker reports local approval using stale policy v17.
<code>manager_review_missing</code>	Event frame	Audit observes the missing manager-review invariant under policy v18.
<code>settlement_blocked</code>	Event frame	Finance blocks downstream settlement.
<code>lc_stale_policy_cache</code>	Latent condition	The policy cache is stale relative to the authoritative policy source.
<code>stales</code>	Causal edge	The latent stale representation produces the false terminal approval.
<code>violates</code>	Causal edge	The false terminal approval violates the manager-review invariant.
<code>fails_downstream</code>	Causal edge	The invariant violation blocks settlement.

6 Why the Result Matters

The result is a compact demonstration of synergy. Lambda-style recursion and DLR ProcessIR are not redundant. They contribute different capabilities:

- recursive control makes the evidence search compositional and bounded;
- typed event frames preserve local status, global status, roles, and invariants;
- the reduce step infers hidden structural preconditions from separated evidence;
- ProcessIR makes the final diagnosis inspectable as a causal graph rather than a paragraph of plausible prose.

The killer observation is that event recall alone is insufficient. The naive merge obtains perfect observed-event recall while still failing the diagnosis. The difference between a log summary and an incident diagnosis is the inferred latent structure.

7 Limitations

This experiment is intentionally small and controlled. The injected evidence uses explicit DLR_LAMBDA markers, so the benchmark does not test open-ended log parsing or natural language extraction. The incident is synthetic, although it is embedded in a real Loghub Linux log substrate. The parser is deterministic and schema-specific. It should therefore be read as a falsification probe for the representation and execution plan, not as a claim of production-grade incident diagnosis.

The result nevertheless establishes a useful target for a larger evaluation: remove explicit markers, expand to multiple Loghub datasets, introduce distractor incidents, use learned parsers for chunk-level extraction, and keep the ProcessIR graph as the adjudication target.

8 Conclusion

The DLR-lambda Loghub experiment shows that recursive execution and typed causal IR are complementary. Recursive split/map/filter/reduce control finds and composes distributed evidence. ProcessIR preserves the distinctions that matter for incident diagnosis: false terminal states, local versus global completion, latent stale representations, violated invariants, root causes, and corrective prescriptions.

In the implemented probe, the direct parser fails, the naive event merge sees events but misses the root cause, and the DLR-lambda reducer recovers the full causal diagnosis. This is the central result: a system can read all the right log lines and still fail unless it has an intermediate representation capable of expressing hidden structural preconditions.

A Appendix: Minimal DLR Context

The broader DLR project is not assumed to be published prior work in this paper. This appendix provides only the context needed to make the standalone experiment understandable.

DLR decomposes language and traces into motifs, frames, graphs, and process diagnoses. The ProcessIR fields used in this paper descend from the trace-level DLR experiments:

- **false terminal state:** a local success report that should not be treated as a valid parent-process terminal state;
- **violated invariant:** a constraint required for valid completion;
- **latent condition:** an inferred precondition or structural decay source not directly present as one event;
- **local/global separation:** distinction between subsystem status and parent-process validity;
- **prescription:** motif-level repair, such as invalidating stale representations or adding reconciliation barriers.

In this experiment, the relevant motifs are Representation, Invariant, Reconciliation, Boundary, and Decay. The latent stale policy cache is a Representation/Decay failure. The missing manager review is an Invariant failure. The blocked settlement is a Reconciliation and terminal-outcome failure.

B Appendix: Reproduction

The experiment is implemented in the repository as:

```
npm run run:dlr-lambda
```

The command downloads the Loghub Linux sample if needed, injects the synthetic incident, writes the Markdown report, and writes the ProcessIR JSON artifact.

References

- [1] Amartya Roy, Rasul Tutunov, Xiaotong Ji, Matthieu Zimmer, and Haitham Bou-Ammar. The Y-Combinator for LLMs: Solving Long-Context Rot with λ -Calculus. arXiv:2603.20105v1, 2026. <https://arxiv.org/abs/2603.20105>.
- [2] LogPAI. Loghub: A large collection of system log datasets for AI-driven log analytics. <https://github.com/logpai/loghub>.
- [3] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu. Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. IEEE International Symposium on Software Reliability Engineering (ISSRE), 2023.
- [4] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R. Lyu. A Large-scale Evaluation for Log Parsing Techniques: How Far are We? ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2024.